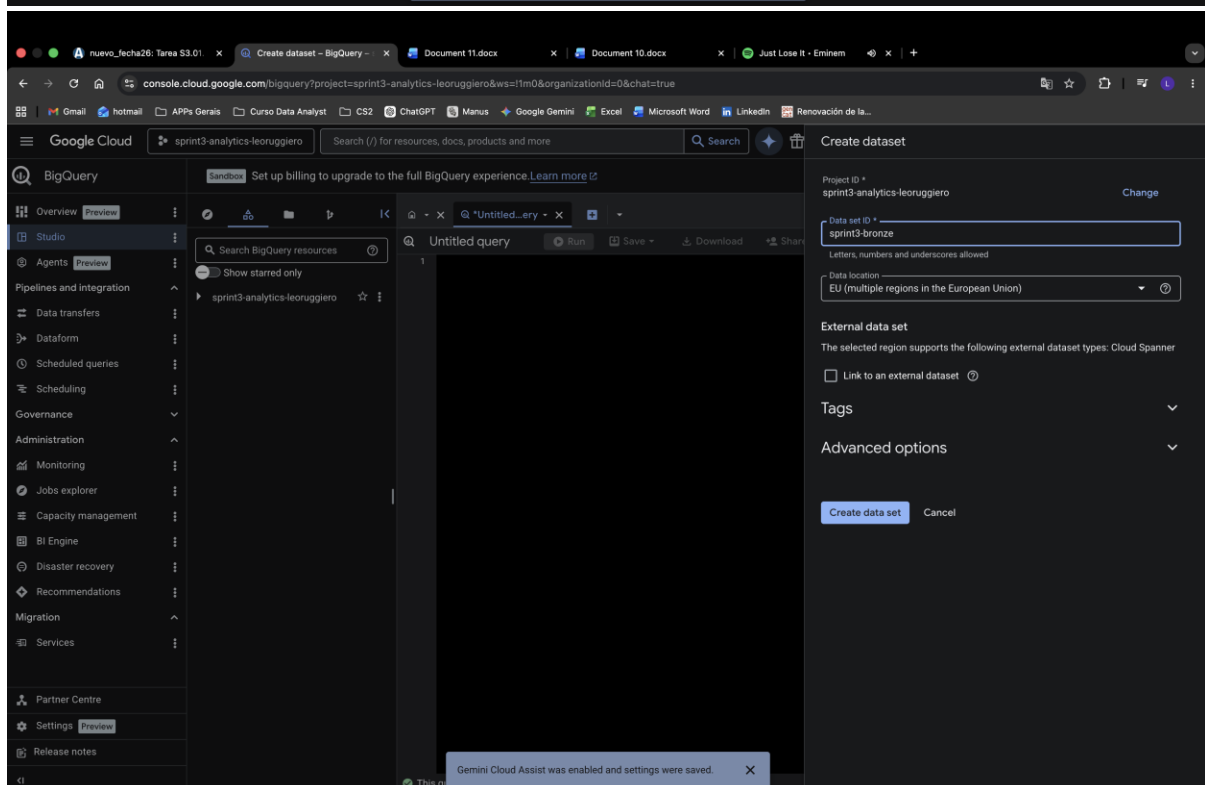
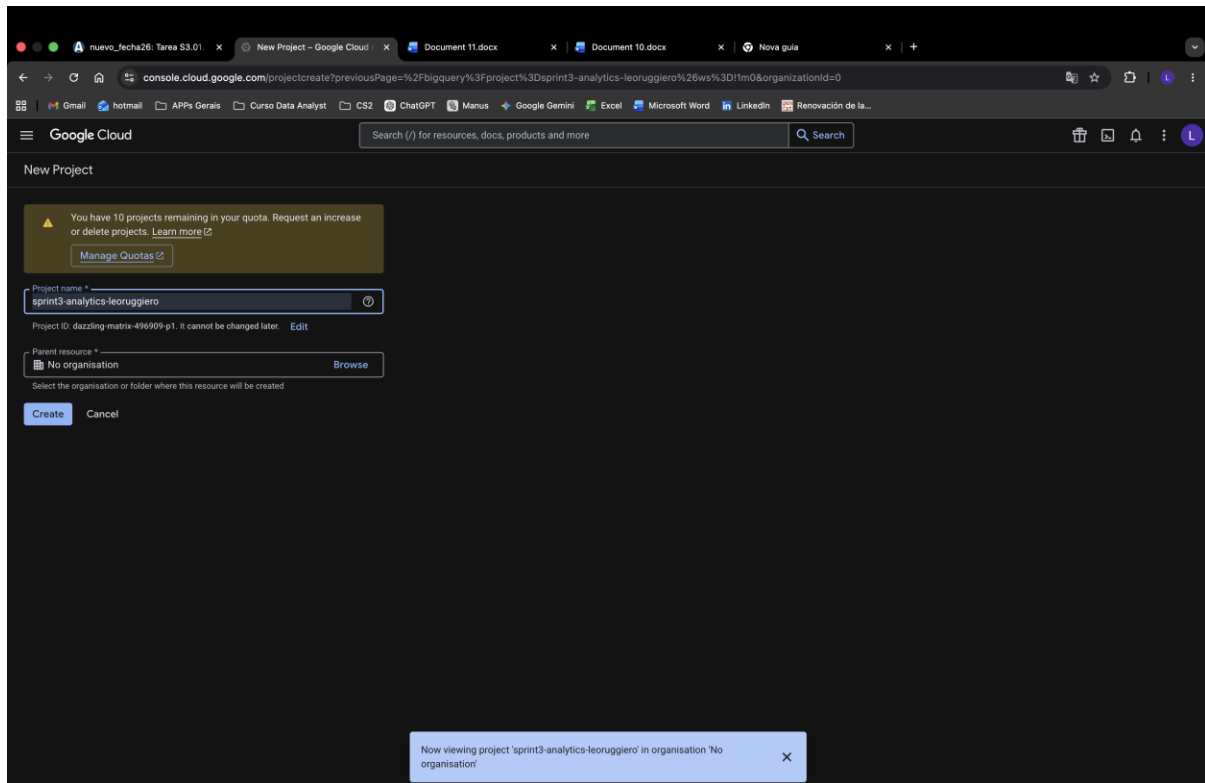


SPRINT 3 – BigQuery 2026 - LEONARDO ANTONIO RUGGIERO

Ejercicio 1: Arquitectura de Datos (Lógica vs. Física)



Google Cloud console showing the BigQuery interface. The left sidebar displays navigation options: Overview, Studio, Agents, Pipelines and integration, Data transfers, Dataform, Scheduled queries, Scheduling, Governance, Administration, Monitoring, Jobs explorer, Capacity management, BI Engine, Disaster recovery, Recommendations, Migration, Services, Partner Centre, Settings, and Release notes. The main panel shows the 'sprint3_analytics-leoruggiero' project with a search bar and a 'Show starred only' filter. The 'sprint3_bronze' dataset is selected, and a table named 'sprint3_bronze' is displayed. A notification at the bottom states 'sprint3_bronze created' with a 'Go to dataset' link.

Table ID	Type	Create time	Expiry time	Label
No rows to display				

Google Cloud console showing the BigQuery interface. The left sidebar displays navigation options: Overview, Studio, Agents, Pipelines and integration, Data transfers, Dataform, Scheduled queries, Scheduling, Governance, Administration, Monitoring, Jobs explorer, Capacity management, BI Engine, Disaster recovery, Recommendations, Migration, Services, Partner Centre, Settings, and Release notes. The main panel shows the 'sprint3_analytics-leoruggiero' project with a search bar and a 'Show starred only' filter. The 'sprint3_bronze' dataset is selected, and a table named 'sprint3_bronze' is displayed. A notification at the bottom states 'sprint3_bronze created' with a 'Go to dataset' link.

```
1 CREATE SCHEMA 'sprint3_silver'
2 OPTIONS(
3   location = 'EU'
4 );
5
```

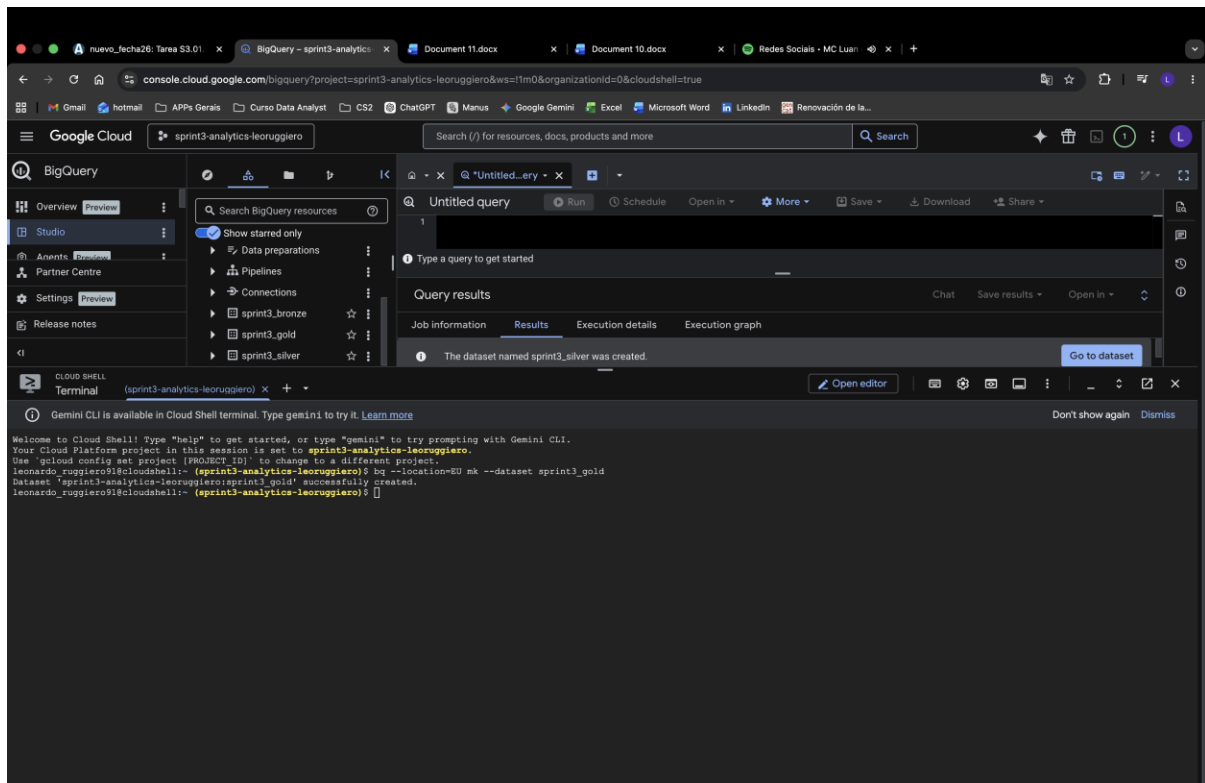
Query completed

Query results

Job information Results Execution details Execution graph

The dataset named sprint3_silver was created.

Go to dataset



He creado el nuevo proyecto 'sprint3-analytics-leoruggiero'

Sprint 3 Bronze:

Detallé paso a paso y de forma manual la creación del conjunto de datos **en la región de la UE**; para ello, hice clic en los tres puntos junto al proyecto, seleccioné 'create dataset' y añadí los valores correspondientes.

Sprint 3 Silver:

Implementada mediante sentencias DDL (SQL CREATE SCHEMA) **configurando la región en la UE**, lo que permite la automatización de la infraestructura como código.

Sprint 3 Gold:

Desplegada a través de la interfaz de línea de comandos (CLI bq) **especificando la región de la UE**, creada con la terminal.

Ejercicio 2: Ingesta en Capa Bronze (Conexión DDL)

The screenshot displays the Google Cloud BigQuery console interface. On the left, the 'sprint3_analytics-leoruggiero' project is selected, showing a hierarchy of resources including 'sprint3_bronze'. The main panel shows an 'Untitled query' with the following SQL code:

```
1 CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.transactions_raw`  
2 OPTIONS (  
3   format = 'CSV',  
4   uris = ['gs://bootcamp-data-analytics-public/ERP/transactions.csv'],  
5   field_delimiter = ',',  
6   skip_leading_rows = 1  
7 );  
8  
9 CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.companies_raw` (  
10  id STRING,  
11  company_name STRING,  
12  address STRING,  
13  city STRING,  
14  country STRING,  
15  phone STRING  
16 );  
17 OPTIONS (  
18   format = 'CSV',  
19   uris = ['gs://bootcamp-data-analytics-public/ERP/companies.csv'],  
20   field_delimiter = ',',  
21   skip_leading_rows = 1  
22 );  
23  
24 CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.american_users_raw` (  
25  id STRING,  
26  first_name STRING,  
27  last_name STRING,  
28  email STRING,  
29  phone STRING,  
30  address STRING,  
31  city STRING,  
32  country STRING,  
33  postal_code STRING  
34 );  
35 OPTIONS (  
36   format = 'CSV',  
37   uris = ['gs://bootcamp-data-analytics-public/ERP/american_users.csv'],  
38   field_delimiter = ',',  
39   skip_leading_rows = 1  
40 );  
41  
42 CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.european_users_raw` (  
43  id STRING,  
44  first_name STRING,  
45  last_name STRING,  
46  email STRING,  
47  phone STRING,  
48  address STRING,  
49  city STRING,  
50  country STRING,  
51  postal_code STRING  
52 );  
53 OPTIONS (  
54   format = 'CSV',  
55   uris = ['gs://bootcamp-data-analytics-public/ERP/european_users.csv'],  
56   field_delimiter = ',',  
57   skip_leading_rows = 1  
58 );
```

Below the query editor, a message states: "NOTE: Could not compute bytes processed estimate for script. Processing location: EU". The 'All results' section shows the query execution details:

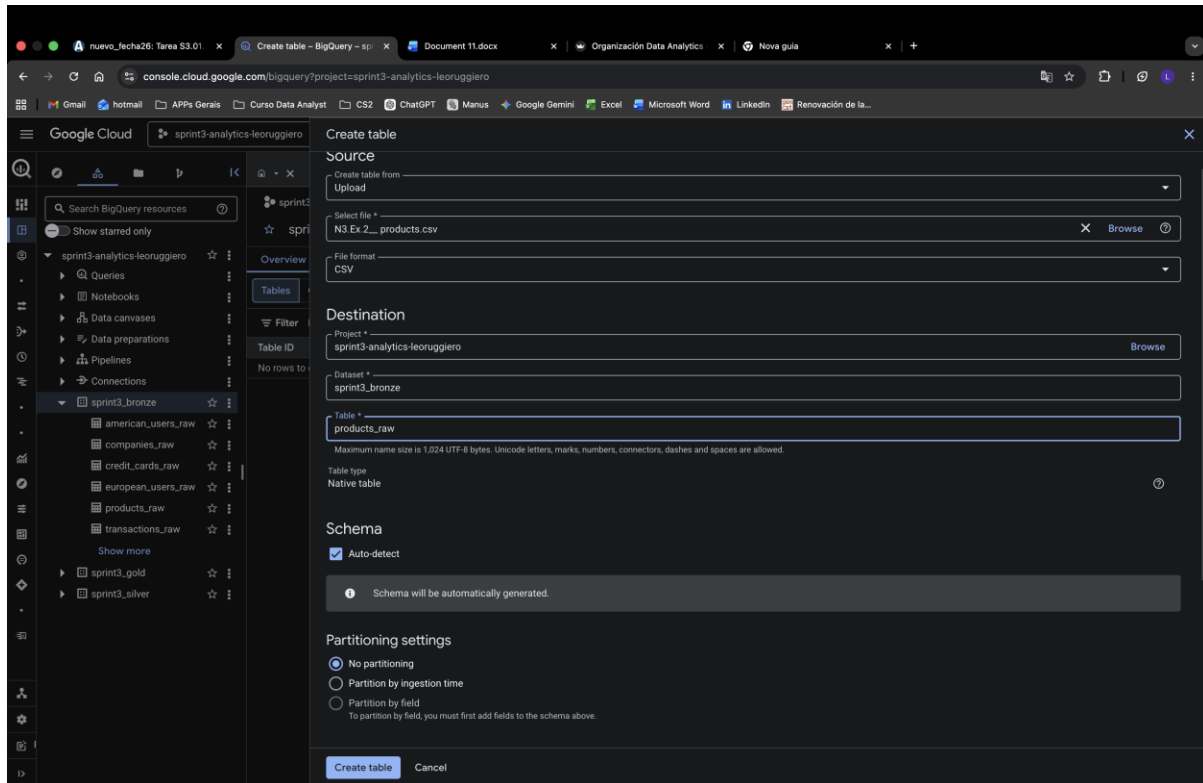
Elapsed time	Statements processed	Job status
4 sec	5	SUCCESS

The 'All results' table also displays a list of statements executed:

Status	End time	SQL	Stages completed	Bytes processed	Action
✓	12:51 [1:1]	CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.transactions_raw`	0	0 B	View results
✓	12:51 [9:1]	CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.companies_raw`	0	0 B	View results
✓	12:51 [24:1]	CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.american_users_raw`	0	0 B	View results
✓	12:51 [42:1]	CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.european_users_raw`	0	0 B	View results
✓	12:51 [60:1]	CREATE OR REPLACE EXTERNAL TABLE `sprint3_bronze.credit_cards_raw`	0	0 B	View results

Una vez configurado el entorno, el siguiente paso fue "conectar" los datos. En lugar de copiar y subir todos los archivos, lo cual consume tiempo y espacio, utilicé una técnica llamada Tablas Externas. Esto significa que le di una instrucción a BigQuery para que "mire" directamente los archivos que ya estaban guardados en el servidor de la empresa (Cloud Storage). Es como crear un acceso directo: los datos se quedan donde están, pero yo puedo consultarlos desde BigQuery como si estuvieran dentro de mi base de datos.

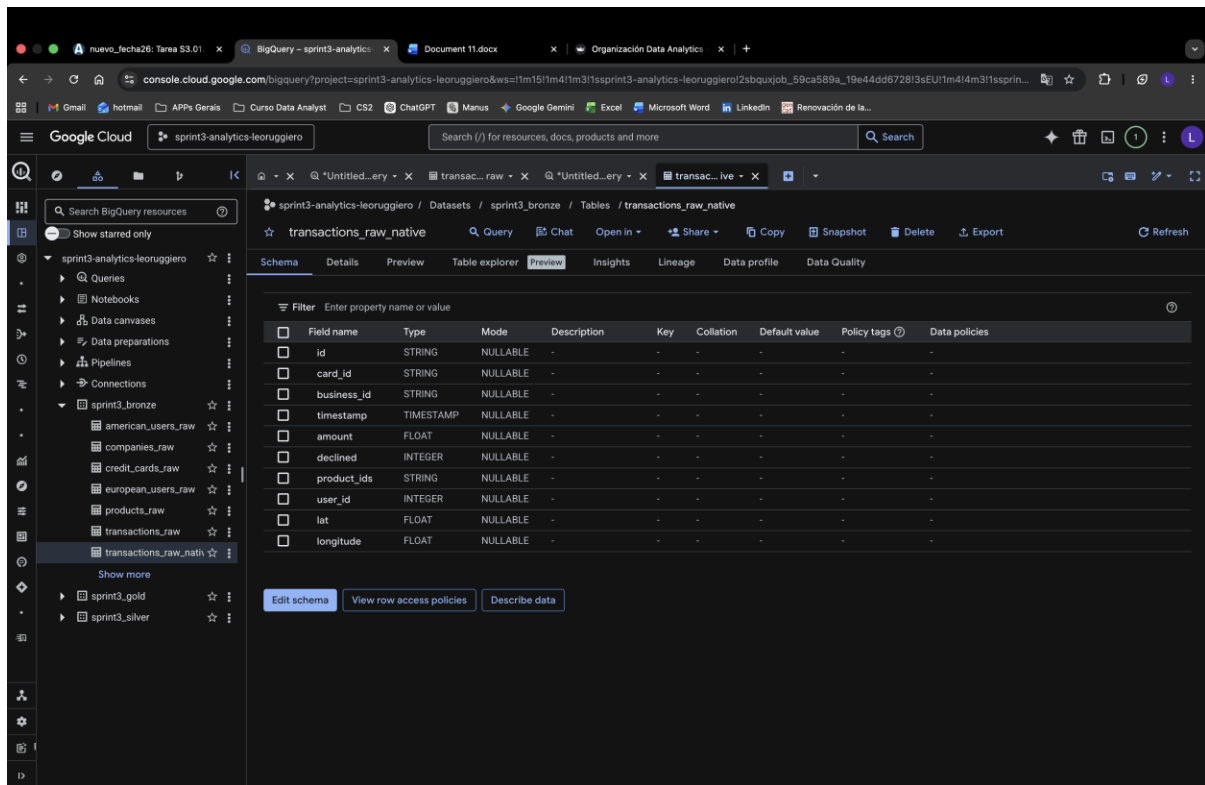
Ejercicio 3: Carga de Datos Locales (Upload)



Para empezar, fui al menú de la izquierda en el Explorador y localicé mi conjunto de datos llamado `sprint3_bronze`. Hice clic en los tres puntos verticales que aparecen justo al lado y seleccioné la opción **Create table**. En el panel que se desplegó en la parte derecha de la pantalla, fui a la sección de **Source** y cambié la opción de **Create table from** a **Upload**. Después, hice clic en **Browse**, busqué en mi computadora el archivo `products.csv` que ya había usado en el Sprint 2 y lo seleccioné, verificando que el formato del archivo estuviera configurado como **CSV**. En la sección de **Destination**, fui al campo **Table** y escribí el nombre `products_raw` para la nueva tabla. Para terminar con la configuración, me aseguré de marcar la casilla de **Auto detect** en el apartado de **Schema** para que el sistema identificara de forma automática los nombres de las columnas y sus tipos de datos, y finalmente hice clic en el botón azul de abajo que dice **Create table**.

Ejercicio 4: Arquitectura y Rendimiento. Materialización de Datos (Asistido por IA)

a) Materialización de Datos (Asistido por IA)



A) En este primer ejercicio no hay mayor explicación; simplemente seguí los pasos utilizando la IA, la cual generó una función que creó el código de creación de la tabla y también definió las columnas de forma automática. Solo fue necesario hacer clic en INSERT y el resultado se completó con éxito.

b) Auditoría de Costes

The screenshot shows the Google Cloud BigQuery console interface. On the left, a sidebar lists various resources like Queries, Notebooks, Data canvases, etc. The main area displays an 'Untitled query' with the following SQL code:

```
1 SELECT id FROM `sprint3-analytics-leoruggiero.sprint3_bronze.transactions_raw`;  
2  
3 SELECT id FROM `sprint3-analytics-leoruggiero.sprint3_bronze.transactions_raw_native`;  
4
```

Below the query editor, the execution status is shown as 'SUCCESS' with an elapsed time of 4 seconds and 2 statements processed. A table titled 'All results' provides a detailed breakdown of the query execution:

Status	End time	SQL	Stages completed	Bytes processed	Action
✓	13:20 [1:1]	SELECT id FROM `sprint3-analytics-leoruggiero.sprint3_bronze.transactions_raw`	2	12.61 MB	View results
✓	13:20 [3:1]	SELECT id FROM `sprint3-analytics-leoruggiero.sprint3_bronze.transactions_raw_native`	2	3.62 MB	View results

B) Cuánto "pesa" la consulta en transactions_raw?

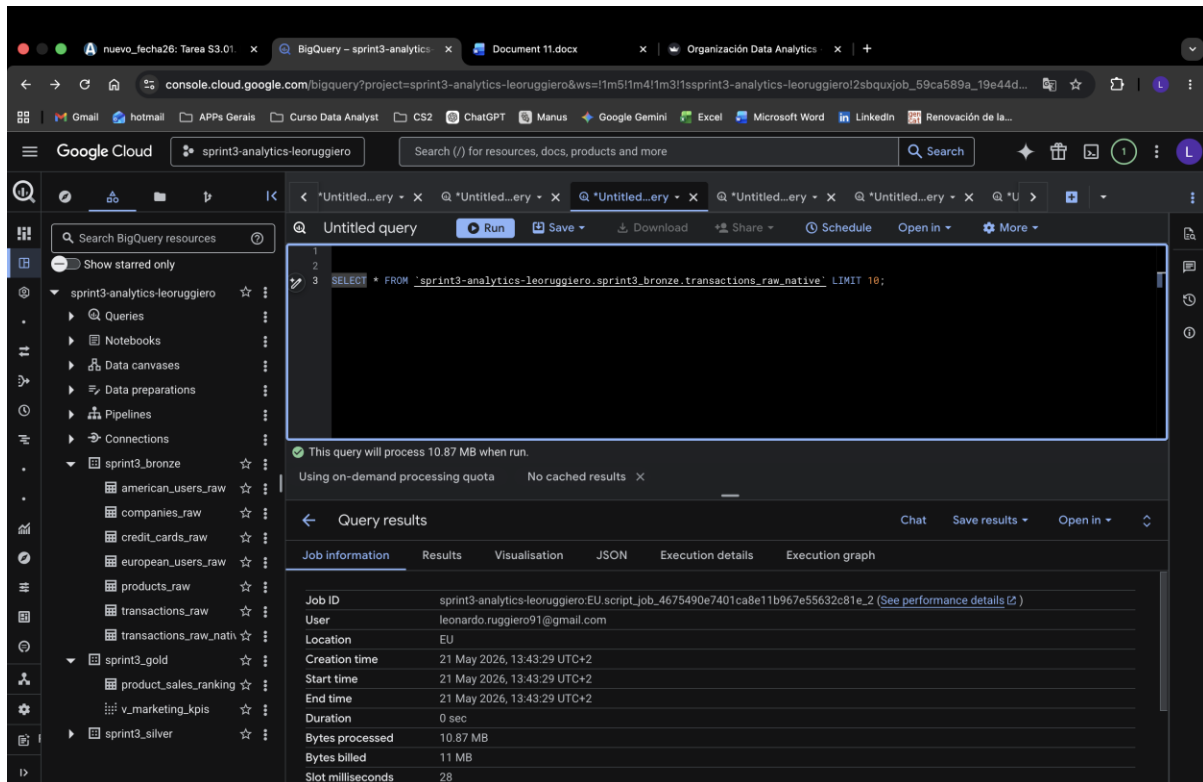
Bytes processed: 12.61 MB

¿Cuánto "pesa" la misma consulta en transactions_raw_native?

Bytes processed: 3.62 MB

En resumen: La tabla nativa corre más rápido simplemente porque BigQuery no tiene que "viajar" hasta un archivo fuera de él para buscar los datos cada vez que presionas el botón. El dato ya juega "en casa". En cambio, la tabla no nativa (externa) es más lenta porque BigQuery necesita "salir de casa", acceder a un archivo que está guardado fuera de él (como en Google Cloud Storage) y leer todo desde cero siempre que haces una consulta. Es como tener que ir al supermercado cada vez que necesitas un ingrediente, en vez de ir a buscarlo a tu propia despensa.

c) El peligro del LIMIT



C) Ha bajado el coste respecto a no poner LIMIT?

No, no ha bajado absolutamente nada. En BigQuery, poner LIMIT no reduce el coste ni los bytes procesados cuando usas SELECT *.

El motor de BigQuery funciona por columnas: al poner SELECT *, te cobra por leer todas las columnas de la tabla entera, sin importar si al final decides mostrar solo 10 filas o un millón. El filtro de filas (LIMIT) se aplica *después* de haber pagado el coste de leer los datos.

La única excepción: El coste solo baja si filtras por columnas (ej. SELECT id) o si la tabla está particionada/clústerizada y usas un filtro WHERE.

Ejercicio 5: Adaptación de Sintaxis (Reporting)

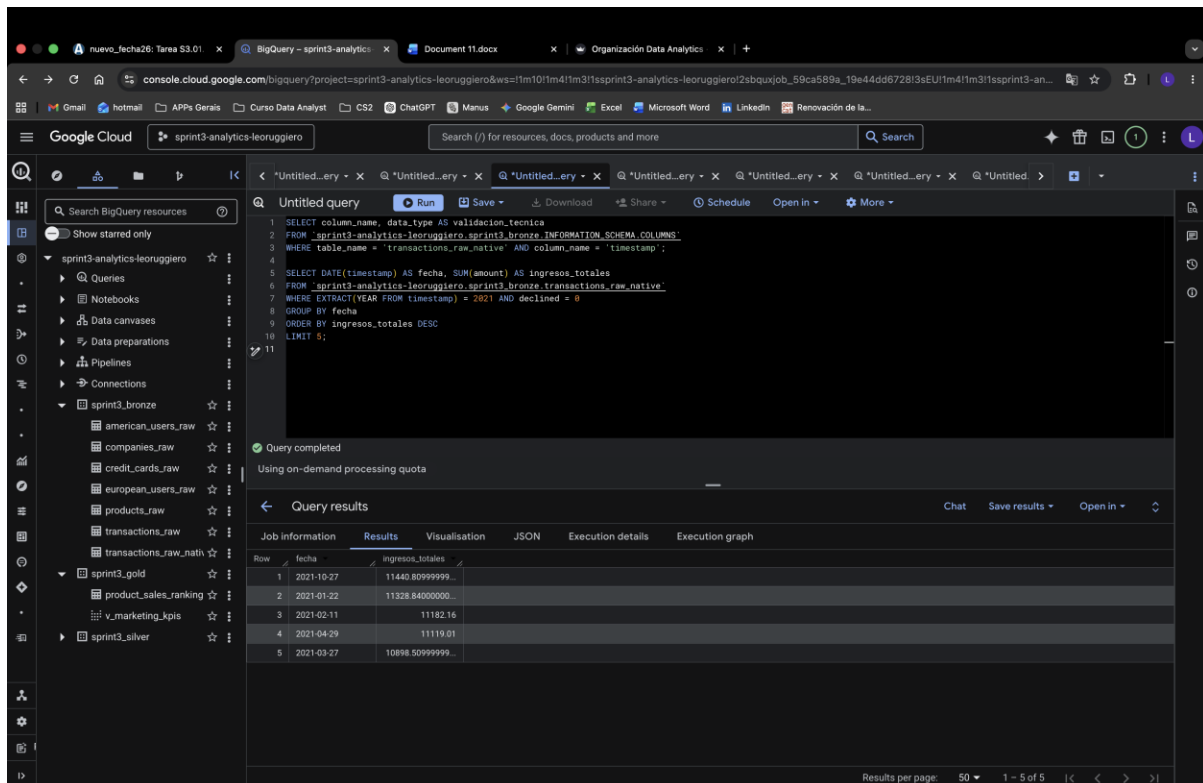
The screenshot displays the Google Cloud BigQuery console interface. The left sidebar shows a project named 'sprint3-analytics-leoruggiero' with various resources like Queries, Notebooks, and Data canvases. The main area shows an 'Untitled query' editor with the following SQL code:

```
1  
2 SELECT column_name, data_type AS validation_tecnica  
3 FROM `sprint3-analytics-leoruggiero.sprint3_bronze.INFORMATION_SCHEMA.COLUMNS`  
4 WHERE table_name = 'transactions_raw_native' AND column_name = 'timestamp';  
5  
6  
7  
8  
9
```

Below the editor, a message states: 'This query will process 10 MB when run. Using on-demand processing quota'. The 'Query results' section is active, showing a table with the following data:

Job information	Results	Visualisation	JSON	Execution details	Execution graph
Row	column_name	validation_tecnica			
1	timestamp	TIMESTAMP			

At the bottom right, it indicates 'Results per page: 50' and '1 - 1 of 1'.



La primera función la utilicé exclusivamente como una validación técnica. Su único objetivo era entrar al sistema (INFORMATION_SCHEMA) y comprobar qué tipo de formato tenía la columna *timestamp* (si era un texto o una fecha real), para así saber exactamente qué funciones de BigQuery debía aplicar después.

La segunda función fue:

Filtrar los datos para quedarse solo con el año 2021 y declined = 0

Agrupar y sumar los ingresos (amount) por cada día.

Ordenar los resultados de mayor a menor y usar un LIMIT 5 para mostrar únicamente los 5 días con mayores ingresos del año.

Ejercicio 6: Consultas Complejas

The screenshot shows the Google Cloud BigQuery console interface. On the left is a sidebar with a search bar and a tree view of resources including 'sprint3-analytics-leoruggiero', 'sprint3_bronze', and 'sprint3_gold'. The main area displays a SQL query in a text editor. Below the query, a status message indicates it will process 3.05 MB. The 'Query results' section is active, showing a table with 7 rows and 4 columns: 'Row', 'nombre_empresa', 'pais', and 'fecha_transaccion'. The results are as follows:

Row	nombre_empresa	pais	fecha_transaccion
1	Dictum Eu Corp.	Canada	2015-04-29 13:46:12 UTC
2	Auctor Mauris Vel LLP	United States	2015-04-29 11:04:13 UTC
3	Tempor Diam Institute	Netherlands	2018-07-20 05:47:38 UTC
4	Tempor Diam Institute	Netherlands	2018-07-20 21:19:49 UTC
5	Et Magnis Ltd	Belgium	2018-07-20 23:32:19 UTC
6	Justo Eu Arcu Ltd	Italy	2018-07-20 11:30:50 UTC
7	Netus Et Malesuada Ltd	Netherlands	2018-07-20 07:25:11 UTC

He diseñado la consulta para este informe complejo cruzando las tablas de transacciones y empresas.

Detalles técnicos de la solución:

INNER JOIN: Conectamos la tabla de transacciones nativa con la tabla de empresas usando el ID de negocio (business_id = id).

BETWEEN 100 AND 200: Filtramos el rango de importes solicitado.

Declined = 0 para buscar solo operaciones realizadas.

DATE(timestamp) IN (...): Utilizamos una lista de fechas específicas en formato ISO (AAAA-MM-DD) para cumplir con el requisito del informe.

Nivel 2: Limpieza y Transformación (ELT)

Ejercicio 1: Limpieza de Productos (Data Quality)

The screenshot shows the Google Cloud BigQuery console interface. On the left, the 'sprint3-analytics-leoruggiero' project is selected, showing a hierarchy of datasets including 'sprint3_bronze', 'sprint3_gold', and 'sprint3_silver'. The main panel displays a SQL query in the 'Untitled query' editor:

```
1  
2 CREATE OR REPLACE TABLE `sprint3-analytics-leoruggiero.sprint3_silver.products_clean`  
3 AS  
4 SELECT  
5   id AS product_id,  
6   product_name AS name,  
7   CAST(REPLACE(warehouse_id, 'WH-', '')) AS INT64 AS warehouse_id,  
8   CAST(price AS FLOAT64) AS price,  
9   weight  
10 FROM `sprint3-analytics-leoruggiero.sprint3_bronze.products_raw`;  
11  
12  
13  
14
```

Below the query editor, a message states: "Query completed. Using on-demand processing quota." The 'Query results' section shows a notification: "This statement created a new table named products_clean." with a 'Go to table' button.

The screenshot shows the Google Cloud BigQuery console interface, specifically the 'Schema' tab for the 'products_clean' table. The table is located in the 'sprint3-analytics-leoruggiero / Datasets / sprint3_silver / Tables / products_clean' path. The schema is displayed as follows:

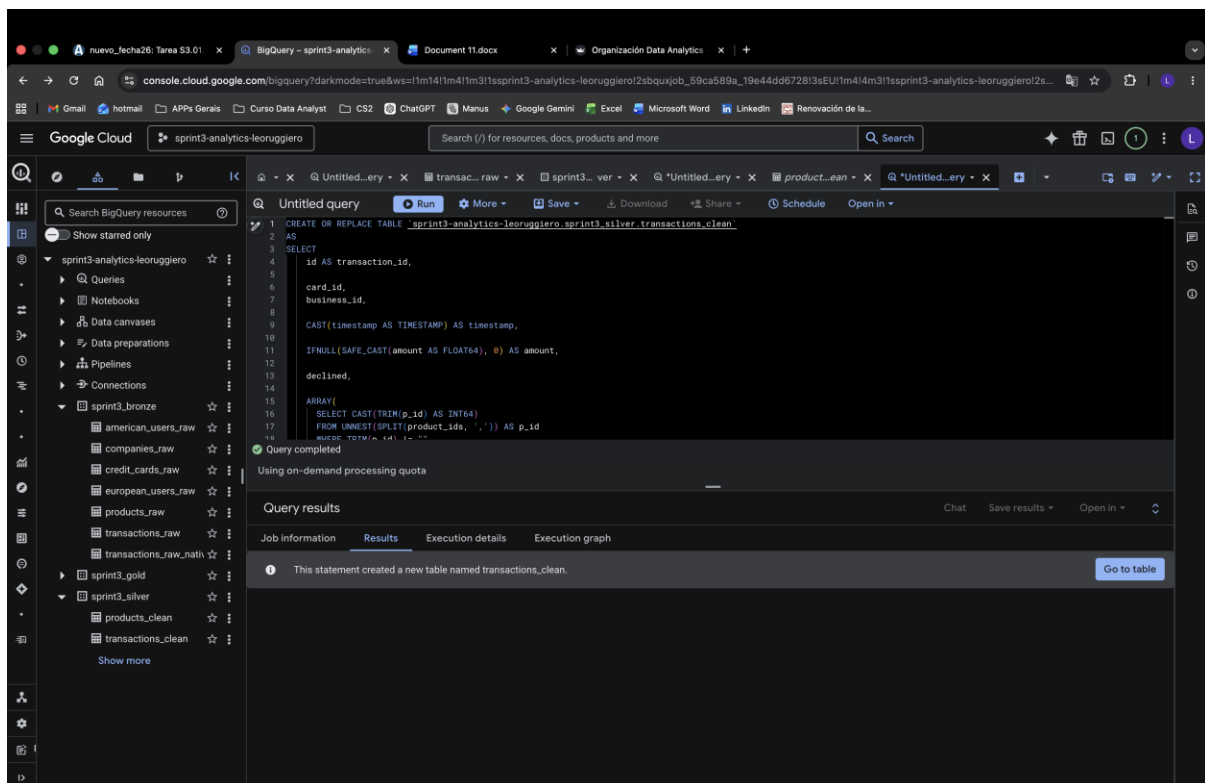
Field name	Type	Mode	Description	Key	Collation	Default value	Policy tags	Data policies
product_id	INTEGER	NULLABLE	-	-	-	-	-	-
name	STRING	NULLABLE	-	-	-	-	-	-
warehouse_id	INTEGER	NULLABLE	-	-	-	-	-	-
price	FLOAT	NULLABLE	-	-	-	-	-	-
weight	FLOAT	NULLABLE	-	-	-	-	-	-

Below the schema table, there are buttons for 'Edit schema', 'View row access policies', and 'Describe data'.

He preparado el script para crear la capa Silver de productos siguiendo todas las reglas de calidad de datos solicitadas:

1. Estandarización: He renombrado id a product_id y product_name a name.
2. Limpieza de IDs: He utilizado REPLACE para eliminar el prefijo "WH-" y luego un CAST para convertir el resultado en un número entero (INT64).
3. Garantía de Precio: He asegurado que el campo sea FLOAT64 mediante un CAST explícito.
4. Preservación: El campo weight se mantiene sin cambios.

Ejercicio 2: Creación de Transacciones Limpias (Capa Silver)



The screenshot shows the Google Cloud BigQuery console. The left sidebar displays a file explorer with a tree structure of datasets and tables. The main area shows the 'transactions_clean' table in the 'sprint3_silver' dataset. The table has 7 columns: transaction_id, card_id, business_id, timestamp, amount, declined, and product. The table preview shows 6 rows of data.

Row	transaction_id	card_id	business_id	timestamp	amount	declined	product
1	26904E68-80E1-41DF-9ECB-3A...	CcS-4857	b-2406	2016-10-12 12:27:29 UTC	6.9	0	
2	A7889200-FEDE-4540-8B55-8B...	CcS-4857	b-2550	2019-06-28 23:40:47 UTC	257.67	0	
3	2C8531E2-AB1C-456F-9A87-BC...	CcS-4857	b-2466	2021-01-18 20:41:16 UTC	482.91	0	
4	FD97C4C4-F01C-4D9D-AE1E-11...	CcS-4857	b-2354	2017-12-29 16:19:35 UTC	318.8	0	
5	856E9267-F16B-46E1-AA9E-90...	CcS-4857	b-2550	2019-10-22 01:42:42 UTC	301.7	0	
6	E69DE824-5156-4EBB-82C7-30...	CcS-4857	b-2346	2021-02-14 23:54:01 UTC	264.24	0	

En este ejercicio, he aplicado varias reglas de calidad de datos para transformar la tabla bruta en una tabla limpia de nivel Silver. Primero, he normalizado los nombres de las columnas para que sean más descriptivos. Para garantizar la robustez del campo amount, he implementado SAFE_CAST junto con IFNULL, asegurando que el flujo de datos no se detenga por errores de formato y que los valores no numéricos se traten como cero.

Uno de los puntos clave de mi desarrollo ha sido la gestión de la columna product_ids. He convertido lo que originalmente era una cadena de texto simple en una estructura de datos tipo ARRAY. Para lograr esto, he utilizado la función SPLIT para separar los elementos y UNNEST para limpiar y convertir cada ID a entero de forma individual. Finalmente, he asegurado que tanto la fecha como las coordenadas geográficas tengan tipos de datos nativos (TIMESTAMP y FLOAT64), lo que permitirá realizar análisis espaciales y temporales mucho más rápidos y precisos.

Ejercicio 3: Unificación de Usuarios (UNION)

The screenshot shows the Google Cloud BigQuery console. On the left, a sidebar lists various datasets under the project 'sprint3-analytics-leoruggiero', including 'sprint3_bronze', 'sprint3_gold', and 'sprint3_silver'. The main area displays a SQL query in a text editor. The query is designed to create a new table 'sprint3-silver.users_combined' by combining data from two existing tables: 'sprint3-analytics-leoruggiero.sprint3_bronze.american_users_raw' and 'sprint3-analytics-leoruggiero.sprint3_bronze.european_users_raw'. The query uses a UNION ALL statement to merge the two datasets. The columns selected are user_id, name, surname, phone, email, birth_date, country, city, postal_code, and address. The origin is specified as 'USA' for the first dataset and 'EUROPE' for the second. The query is ready to be executed, with a status bar indicating it will process 0 B of data.

```
1 CREATE OR REPLACE TABLE `sprint3-analytics-leoruggiero.sprint3_silver.users_combined` AS
2 SELECT
3   id AS user_id,
4   name,
5   surname,
6   phone,
7   email,
8   birth_date,
9   country,
10  city,
11  postal_code,
12  address,
13  'USA' AS origin
14 FROM
15   `sprint3-analytics-leoruggiero.sprint3_bronze.american_users_raw`
16 UNION ALL
17 SELECT
18   id AS user_id,
19   name,
20   surname,
21   phone,
22   email,
23   birth_date,
24   country,
25   city,
26   postal_code,
27   address,
28   'EUROPE' AS origin
29 FROM
30   `sprint3-analytics-leoruggiero.sprint3_bronze.european_users_raw`;
```

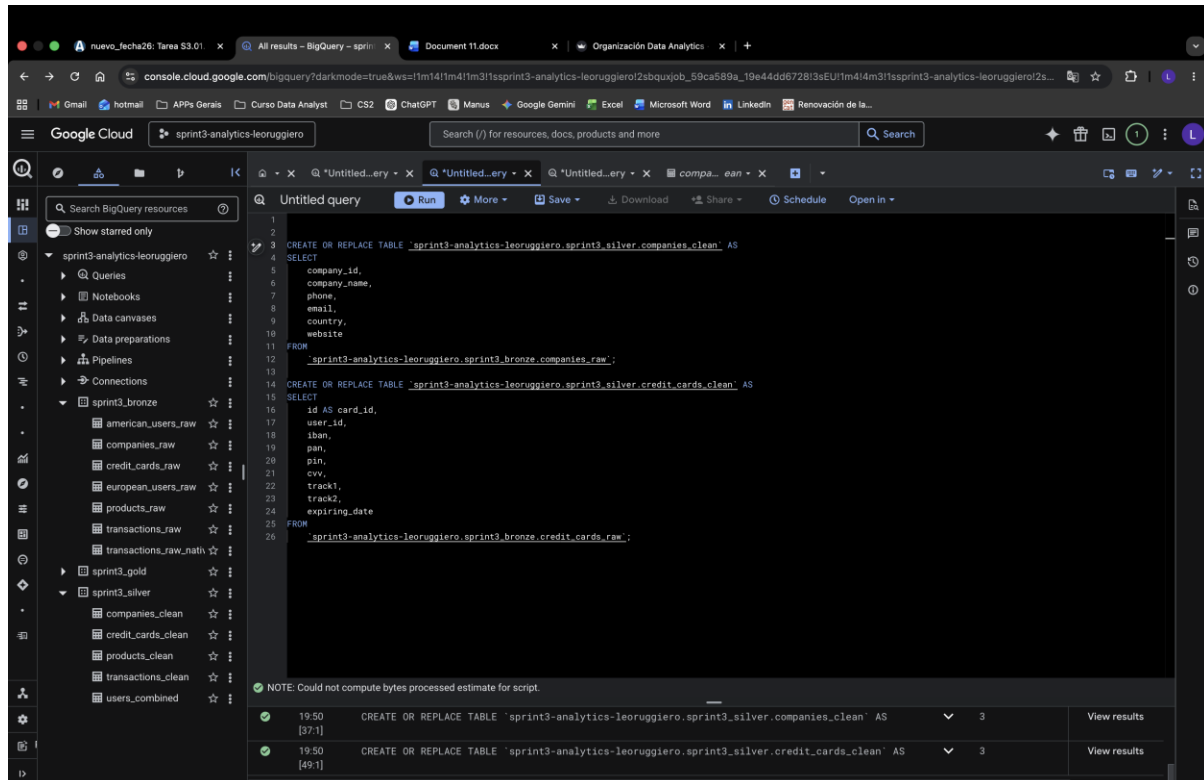
The screenshot shows the Google Cloud BigQuery console displaying the 'users_combined' table. The table is located within the 'sprint3_silver' dataset. The 'Preview' tab is selected, showing a list of 19 rows. The columns are user_id, name, surname, phone, email, and birth_date. The data is a combination of users from the 'american_users_raw' and 'european_users_raw' datasets.

Row	user_id	name	surname	phone	email	birth_date
1	907	Zwtyhn	Solsqik	+94-191-7839	zwtyhn.solsqik@example.com	Jan 3, 1996
2	3904	Ugtusx	Aoxdiemv	+65-997-4681	ugtusx.aoxdiemv@example.com	Jan 6, 1953
3	723	Wdgzjh	Wiqijhfn	+62-276-1153	wdgzjh.wiqijhfn@example.com	Apr 23, 1976
4	1405	Chjqfw	Pzbfjvhp	+61-591-8811	chjqfw.pzbfjvhp@example.com	Jul 24, 1975
5	2013	Erswn	Ohjzsch	+75-206-6920	erswn.ohjzsch@example.com	Nov 8, 1975
6	4438	Wypaxn	Abwzprww	+40-290-2892	wypaxn.abwzprww@example.c...	Sep 8, 1992
7	1772	Rysxpk	Jkfeheuh	+45-547-8984	rysxpk.jkfeheuh@example.com	Aug 21, 2002
8	895	Pimesu	Jzyalbpd	+36-755-3737	pimesu.jzyalbpd@example.com	May 14, 1998
9	2419	Akezzx	Gyjheyuv	+66-453-6021	akezzx.gyjheyuv@example.com	Apr 21, 1963
10	4281	Btxrf	Lqkukipk	+81-202-4145	btxrf.lqkukipk@example.com	Sep 12, 1966
11	238	Chester	Haynes	059-783-3104	ut@protonmail.edu	Sep 13, 1988
12	1284	Hhbred	Extswwhr	+74-147-2392	hbred.extswwhr@example.com	Mar 14, 1975
13	1760	Gmthwp	Ioxltfyv	+56-494-8132	gmthwp.ioxltfyv@example.com	Feb 14, 1986
14	3739	Dfcwtz	Gnsombfd	+92-981-9785	dfcwtz.gnsombfd@example.com	Jun 20, 1973
15	3810	Kfahfj	Zfshhyc	+64-488-5469	kfahfj.zfshhyc@example.com	Apr 14, 2000
16	4058	Rsigkq	Mhmabmpe	+63-952-1552	rsigkq.mhmabmpe@example.c...	Feb 11, 1973
17	1401	Devnbq	Zlqgskuu	+71-537-6029	devnbq.zlqgskuu@example.com	Sep 22, 1950
18	3229	Rowwvz	Rbdxkogb	+98-260-4424	rowwvz.rbdxkogb@example.com	Dec 9, 1963
19	4202	Kiucxi	Razjwlfw	+50-692-1035	kiucxi.razjwlfw@example.com	Aug 3, 1989

Para el ejercicio tres que pide la unificación de los usuarios yo creé la tabla llamada sprint3_silver.users_combined utilizando el comando create or replace table para

estructurar esta nueva lista directamente en la capa silver y para juntar los datos fragmentados de las dos regiones usé el operador union all combinando los registros que venían de la tabla bronze de los estados unidos con los registros de la tabla bronze de europa garantizando que ningún dato fuera descartado y dentro de cada bloque de select añadí una columna calculada llamada origin definiendo el texto usa para los americanos y el texto europe para los europeos lo que permite identificar la procedencia exacta de cada fila en la lista maestra final además para cumplir el requisito de estandarización de nombres y eliminar la ambigüedad de la clave primaria apliqué un alias cambiando el nombre de la columna original id por user_id justo al principio de la selección y mantuve los otros campos como nombre apellido teléfono correo electrónico fecha de nacimiento país ciudad código postal y dirección alineados exactamente en el mismo orden en ambos lados para que el bigquery hiciera la unión perfectamente y sin errores de compatibilidad generando así una tabla limpia y centralizada con todos los usuarios del crm.

Ejercicio 4: Materialización de Compañías y Tarjetas de Crédito



The screenshot displays the Google Cloud BigQuery console interface. On the left, a sidebar shows the project hierarchy for 'sprint3-analytics-leoruggiero', including tables like 'american_users_raw', 'companies_raw', 'credit_cards_raw', 'european_users_raw', 'products_raw', 'transactions_raw', and 'users_combined'. The main area shows a SQL query being executed. The query consists of two parts: creating a table 'sprint3-analytics-leoruggiero.sprint3_silver.companies_clean' AS and creating a table 'sprint3-analytics-leoruggiero.sprint3_silver.credit_cards_clean' AS. The first part selects columns from 'sprint3-analytics-leoruggiero.sprint3_bronze.companies_raw'. The second part selects columns from 'sprint3-analytics-leoruggiero.sprint3_bronze.credit_cards_raw'. Below the query, the execution results are shown, indicating that the tables were created successfully.

```
1 CREATE OR REPLACE TABLE `sprint3-analytics-leoruggiero.sprint3_silver.companies_clean` AS
2
3 SELECT
4   company_id,
5   company_name,
6   phone,
7   email,
8   country,
9   website
10
11 FROM
12   `sprint3-analytics-leoruggiero.sprint3_bronze.companies_raw`;
13
14 CREATE OR REPLACE TABLE `sprint3-analytics-leoruggiero.sprint3_silver.credit_cards_clean` AS
15 SELECT
16   id AS card_id,
17   user_id,
18   iban,
19   pan,
20   pin,
21   cvv,
22   track1,
23   track2,
24   expiring_date
25 FROM
26   `sprint3-analytics-leoruggiero.sprint3_bronze.credit_cards_raw`;
```

Query ID	Query Text	Rows	View results
19:50 [37:1]	CREATE OR REPLACE TABLE `sprint3-analytics-leoruggiero.sprint3_silver.companies_clean` AS	3	View results
19:50 [49:1]	CREATE OR REPLACE TABLE `sprint3-analytics-leoruggiero.sprint3_silver.credit_cards_clean` AS	3	View results

Google Cloud BigQuery console view of the `sprint3-analytics-leoruggiero` project, specifically the `sprint3_silver` dataset and `companies_clean` table. The table is displayed in 'Preview' mode, showing 19 rows of data. The columns are `company_id`, `company_name`, `phone`, `email`, `country`, and `website`.

Row	company_id	company_name	phone	email	country	website
1	b-2498	Metus Vitae Associates	08 25 44 40 66	ad@protonmail.com	Australia	https://
2	b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://
3	b-2446	Risus Associates	04 25 82 54 31	egestas.ligula.nullam@google.e...	Australia	https://
4	b-2534	Mattis Foundation	06 74 03 95 74	sed@google.couk	Australia	https://
5	b-2334	Amet Institute	06 33 40 21 33	nullam.lobortis.quam@outlook...	Australia	https://
6	b-2282	Pretium Neque Corp.	07 77 48 55 28	eleifend.nec.malesuada@proto...	Australia	https://
7	b-2598	Aliquam Iaculis Lacus Corp.	04 43 07 91 26	dictum@aol.org	Belgium	https://
8	b-2462	Pede Ultrices Ltd	04 52 40 26 17	justo.proin@yahoo.com	Belgium	https://
9	b-2514	Cras Consulting	07 50 10 85 63	sed.consequat@google.ca	Belgium	https://
10	b-2594	Et Magnis Ltd	03 53 76 11 76	dapibus.ligula@protonmail.net	Belgium	https://
11	b-2258	Vestibulum Lorem PC	02 02 87 33 40	aenean.massa.integer@aol.net	Belgium	https://
12	b-2314	A Institute	03 34 91 68 65	metus.aliquam@google.edu	Belgium	https://
13	b-2590	Euismod Mauris Institute	02 13 69 54 85	vivamus.molestie@icloud.ca	Belgium	https://
14	b-2274	Sed LLC	01 63 16 26 52	at@outlook.com	Belgium	https://
15	b-2362	Lorem Eu Incorporated	01 83 66 62 07	enim.gravida.sit@hotmail.net	Canada	https://
16	b-2606	Sed Est Corp.	04 58 02 37 91	tempor.bibendum@google.couk	Canada	https://
17	b-2298	Elit Etiam Laoreet Associates	07 69 74 17 45	ultrices@google.couk	Canada	https://
18	b-2526	Amet Luctus Vulputate Foundat...	03 18 54 24 19	nulla@outlook.com	Canada	https://
19	b-2562	Dictum Eu Corp.	03 04 73 67 31	donec.vitae@icloud.ca	Canada	https://

Google Cloud BigQuery console view of the `sprint3-analytics-leoruggiero` project, specifically the `sprint3_silver` dataset and `credit_cards_clean` table. The table is displayed in 'Preview' mode, showing 19 rows of data. The columns are `card_id`, `user_id`, `iban`, `pan`, `pin`, `cvv`, and `track1`.

Row	card_id	user_id	iban	pan	pin	cvv	track1
1	CcU-4688	25	IS383255756743611896271082	347635882232257	3159	350	%B4387334195135200*Xi
2	CcU-4835	4	PT34592171131763200132583	3723 677744 22550	1149	680	%B6221188915952608*En
3	CcU-4478	55	SI52116155174087513	5451685219226844	3161	939	%B94532762951691116*Ar
4	CcU-4163	100	SE7951484302898821300669	4556 776 96 5734	7098	953	%B8157441949696177*Wj
5	CcU-3680	169	MD4799721070815687888150	5426 4337 3314 6889	8556	690	%B4117962041553164*Ng
6	CcU-3155	244	AD2777204763277722050982	4532263578421	3015	779	%B3064783482720541*Di
7	CcU-4443	60	BH44392039643136345028	491 65266 67312 570	2283	141	%B2759365205699185*Ph
8	CcU-3120	249	RS7265576666166237144	527646 533375 6577	7658	265	%B3818528147385482*Id
9	CcU-3127	248	PT83533461438644342816864	4716 443 46 4368	8038	924	%B3061281355691188*Xb
10	CcU-3918	135	NO44147577617220	5567675343638684	5616	230	%B8075164763517864*Xo
11	CcU-3603	180	IT166ASAB181164333755123881	376495785724718	3355	862	%B8213382423731002*Lo
12	CcU-3764	157	DO8323918488352565612425...	535 23847 59612 247	6546	194	%B7633657814281284*Gc
13	CcU-3323	220	BG05UBIX63820348156559	4716 5927 4626 2576	8319	860	%B3897618670151179*Uk
14	CcU-4646	31	CR7237735131501258248	4485 8278 9259 2981	1686	987	%B4539812337700467*In
15	CcU-3099	252	PT261052735682370537218	448 55418 98863 789	5612	564	%B587543125326368*Sc
16	CcU-3393	210	VG1468087984174645729577	4338 849 61 6580	7375	763	%B3767361120135239*Ln
17	CcU-3463	200	RS35935488614486812686	4485654846578	7658	260	%B5335456358287213*Im
18	CcU-3386	211	HR9310391623647587418	375233562287679	7448	433	%B5310605594141993*Mc
19	CcU-5242	661	XX39110058539326212369074	1186426439428165	9378	722	%B1186426439428165*FT

el ejercicio cuatro que pide la materialización de compañías y tarjetas de crédito yo creé las dos tablas nativas llamadas `sprint3_silver.companies_clean` y `sprint3_silver.credit_cards_clean` utilizando el comando `create` o `replace table` para asegurar que los datos se guarden físicamente dentro del almacenamiento de bigquery sin depender de los archivos csv externos y para estructurar la tabla de compañías seleccioné los campos originales como `company_name` teléfono `email` país y `website`

manteniendo el identificador como `company_id` porque ya venía bien definido desde el origen mientras que para la tabla de tarjetas de crédito seleccioné campos como `user_id` `iban` `pan` `pin` `cvv` `track1` `track2` y `expiring_date` pero apliqué un alias para cambiar la columna genérica `id` por `card_id` cumpliendo con el requisito de estandarización y eliminando cualquier tipo de ambigüedad en la clave primaria lo que permitió importar toda la información de la capa bronze a la capa silver de forma limpia rápida y con un rendimiento nativo ideal para los análisis finales del modelo de datos.

Nivel 3: Presentación de Datos y Creación de Vistas

Ejercicio 1: La Vista de Marketing (Lógica de Negocio)

The screenshot shows the Google Cloud BigQuery console interface. On the left, a sidebar lists various data resources, including 'sprint3-bronze' and 'sprint3-gold' datasets. The main panel displays a SQL query editor with the following code:

```
1 CREATE OR REPLACE VIEW `sprint3-analytics-leoruggiero.sprint3-gold.v_marketing_kpis`
2 AS
3 SELECT
4   c.company_name,
5   c.phone,
6   c.country,
7   ROUND(AVG(t.amount), 2) AS avg_amount,
8   CASE
9     WHEN AVG(t.amount) > 260 THEN 'Premium'
10    ELSE 'Standard'
11  END AS client_tier
12 FROM `sprint3-analytics-leoruggiero.sprint3-silver.companies_clean` AS c
13 JOIN `sprint3-analytics-leoruggiero.sprint3-silver.transactions_clean` AS t
14   ON c.company_id = t.business_id
15 WHERE t.declined = 0
16 GROUP BY c.company_name, c.phone, c.country;
```

Below the query editor, a message states: "This query will process 0 B when run." The 'Query results' section is visible, showing a message: "This statement replaced the view named v_marketing_kpis." A 'Go to view' button is present.

The screenshot shows the Google Cloud BigQuery console with the same SQL query editor. The query is now executed, and the results are displayed in a table. The table has columns: company_name, phone, country, avg_amount, and client_tier. The results are as follows:

Row	company_name	phone	country	avg_amount	client_tier
1	Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.91	Premium
2	Pretium Neque Corp.	07 77 48 55 28	Australia	275.58	Premium
3	Uma Convalis Associates	06 01 24 77 04	United States	273.57	Premium
4	At Associates	09 56 61 10 65	New Zealand	272.74	Premium

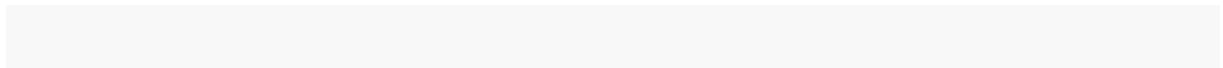
The 'Query results' section also shows a message: "Query completed. Using on-demand processing quota." The 'Results' tab is selected, and the table is displayed. The 'Results per page' is set to 50, and the total number of results is 1 of 100.

Para resolver este ejercicio, tuve que diseñar un reporte automatizado cruzando dos de nuestras tablas que ya habían sido limpiadas en la capa anterior. Primero, seleccioné la tabla de empresas, que contiene datos esenciales como el nombre de la

organización, el teléfono y el país, y la conecté directamente con nuestro historial de ventas. Un detalle importante que apliqué en esta unión fue colocar un filtro para ignorar todas las compras que habían sido rechazadas o canceladas, garantizando así que el cálculo final considerara únicamente el dinero que realmente ingresó.

Con estas dos fuentes unidas, le ordené al sistema que calculara de forma automática el promedio de gastos de cada empresa y, con base en ese valor, creé una regla de negocio para categorizar a los clientes; si el promedio de compra de la empresa es superior a los doscientos sesenta euros, el reporte le asigna automáticamente la etiqueta de cliente "Premium", de lo contrario, el cliente recibe la etiqueta de "Standard". Toda esta lógica quedó guardada bajo el formato de una vista, lo cual es ideal porque funciona como una ventana que se actualiza sola cada vez que entran nuevas ventas en la base de datos.

Por último, ejecuté una consulta general sobre esta vista, configurando el criterio de ordenación para que el reporte muestre primero a todos los clientes de la categoría "Premium" organizados de mayor a menor según su valor promedio de compra, seguidos por los clientes "Standard" bajo la misma lógica, permitiendo que el equipo de Marketing sepa exactamente a quiénes priorizar en un abrir y cerrar de ojos.



Ejercicio 2: Ranking de Productos (La Potencia de los Arrays)

The screenshot shows the Google Cloud BigQuery console interface. On the left, a sidebar displays the project hierarchy for 'sprint3-analytics-leoruggiero', including datasets like 'sprint3_bronze', 'sprint3_gold', and 'sprint3_silver'. The main panel shows an 'Untitled query' editor with the following SQL code:

```
1 CREATE OR REPLACE TABLE `sprint3-analytics-leoruggiero.sprint3_gold.product_sales_ranking` AS
2 SELECT
3   p.product_id,
4   p.name,
5   p.price,
6   IFNULL(COUNT(t.unfolded_product_id), 0) AS total_sold
7 FROM
8   `sprint3-analytics-leoruggiero.sprint3_silver.products_clean` AS p
9 LEFT JOIN (
10  SELECT
11    p_id AS unfolded_product_id
12  FROM
13    `sprint3-analytics-leoruggiero.sprint3_silver.transactions_clean`,
14    UNNEST(product_ids) AS p_id
15  WHERE
16    SAFE_CAST(declined AS STRING) = '0'
17 ) AS t
18 ON SAFE_CAST(p.product_id AS STRING) = SAFE_CAST(t.unfolded_product_id AS STRING)
19 GROUP BY
20   p.product_id,
21   p.name,
22   p.price;
```

Below the query editor, a message states: 'Query completed. Using on-demand processing quota.' The 'Query results' section shows a notification: 'This statement created a new table named product_sales_ranking.' with a 'Go to table' button.

The screenshot shows the same Google Cloud BigQuery console interface, but now displaying the results of the query. The 'Query results' section is expanded, showing a table with the following data:

Row	product_id	name	price	total_sold
1	52	riverlands the duel	31.52	2642
2	29	Tully maester Early	167.2	2627
3	21	duel Direwolf	96.9	2603
4	16	the duel warden	180.91	2602
5	33	duel warden	127.09	2593
6	87	sith Jade	96.26	2591
7	66	mustafar jinn	2.12	2590

At the bottom of the results section, it indicates 'Results per page: 50' and '1 - 50 of 100'.

Para resolver este informe estratégico de ventas, lo que hice fue aprovechar la estructura avanzada de listas de tipo Array que habíamos organizado previamente en la capa Silver, solucionando el reto de que los identificadores de los productos estaban guardados juntos dentro de una misma celda por cada transacción, lo cual impedía contarlos de forma directa tradicional. Para lograrlo, utilicé la función UNNEST sobre la lista de productos en las transacciones exitosas, lo cual se encarga de desempaquetar la celda y generar una fila independiente para cada elemento vendido; sin embargo, al notar que el sistema generaba un error de firmas porque la tabla de catálogo guardaba los IDs como texto mientras que las transacciones los trataban como números enteros, apliqué un filtrado de conversión posicional utilizando la función SAFE_CAST en ambas columnas dentro de la condición del JOIN para estandarizarlas temporalmente como cadenas de texto. Con esta compatibilidad resuelta, realicé un cruce de tipo LEFT JOIN comenzando estrictamente desde nuestra tabla de catálogo de productos limpios, una decisión de ingeniería que fue clave para mantener la integridad del reporte y asegurar que se incluyeran absolutamente todos los productos del inventario, incluyendo aquellos que registran cero ventas en el historial y que representan candidatos ideales para ser descatalogados por falta de rendimiento comercial. Finalmente, agrupé la información por los atributos principales del producto y controlé los valores nulos mediante la instrucción IFNULL para transformarlos en ceros reales, materializando el resultado en una nueva tabla optimizada de la capa Gold que se muestra ordenada de mayor a menor éxito de ventas para facilitar la toma de decisiones del equipo de negocio.

Ejercicio 3: Exportación de Resultados

The screenshot shows the Google Cloud BigQuery console. A query is executed against the `sprint3-gold` dataset, specifically the `product_sales_ranking` table. The query is:

```
1 SELECT * FROM `sprint3-analytics-leoruggiero.sprint3_gold.product_sales_ranking`  
2 ORDER BY total_sold DESC;
```

The query results are displayed in a table with the following columns: `product_id`, `name`, `price`, and `total_sold`. The results are ordered by `total_sold` in descending order.

Row	product_id	name	price	total_sold
1	52	riverlands the duel	31.52	2642
2	29	Tully maester Tarly	167.2	2627
3	21	duel Direwolf	96.9	2603
4	16	the duel warden	180.91	2602
5	33	duel warden	127.09	2593
6	87	sith Jade	96.26	2591
7	66	mustafar jinn	2.12	2590
8	48	rock Renly in	21.53	2589

The screenshot shows a Microsoft Excel spreadsheet with the data exported from the BigQuery query. The data is organized into columns labeled `product_id`, `name`, `price`, and `total_sold`. The data is ordered by `total_sold` in descending order.

product_id	name	price	total_sold
52	riverlands the duel	31.52	2642
29	Tully maester Tarly	167.2	2627
21	duel Direwolf	96.9	2603
16	the duel warden	180.91	2602
33	duel warden	127.09	2593
87	sith Jade	96.26	2591
66	mustafar jinn	2.12	2590
48	rock Renly in	21.53	2589
68	Stark Karstark	167.37	2587
23	riverlands north	169.96	2586
88	Stannis warden south	167.15	2582
73	Dorne bastard	114.09	2579
34	of north	128.48	2574
31	Lannister	85.02	2573
4	warden south duel	71.89	2573
54	dooku solo	13.14	2572
28	chewbacca mustafar	127.44	2572
38	ewok sith	55.43	2571
39	warden	196.65	2570
36	Lannister	96.22	2570
47	Tully	82.15	2568
72	north	161.6	2568
61	Winterfell Lannister	28.01	2568
74	duel warden	158.29	2565
24	south duel tourney	48.99	2564

Primero, ejecuté una consulta directa sobre la tabla `product_sales_ranking` en nuestra capa `Gold`, la cual ya consolida el orden jerárquico de los productos más vendidos. Una vez procesados los resultados en la consola, utilicé la herramienta de extracción

nativa para descargar los datos en un formato estándar compatible con hojas de cálculo locales.

Finalmente, abrí el archivo en un entorno de edición de escritorio, donde apliqué una capa de formato profesional orientada a negocio: ajusté el ancho de las columnas para evitar textos truncados, configuré las métricas financieras bajo el formato de moneda correspondiente y destaqué la fila de encabezados en negrita para facilitar una lectura rápida y estructurada. Este archivo final quedó completamente optimizado para presentarse directamente en reuniones ejecutivas, permitiendo identificar de un vistazo el rendimiento comercial de nuestro catálogo sin necesidad de acceder a la base de datos cloud.